

Homework 2 - DFA and NFA Construction

Ewan Pedersen

2025-02-07

Abstract

This document the formal definitions and constructs for 3 primary DFAs/NFAs, with two extra considered for extra credit. We seek to prove that a language is regular by providing a DFA that recognizes it.

IMPORTANT NOTE: My image rendered is currently a bit broken, and will sometimes jump a page(will rewrite myself soon). For now, refer to figure caption to match images to their respective location.

Definitions

We define a **DFA/NFA** as a 5 tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- Q is a finite **set of states**
 - Σ is a finite **set of symbols**
 - $\delta : Q \times \Sigma \rightarrow Q$ is the **transition function**
 - $q_0 \in Q$ is the **start state**
 - $F \subseteq Q$ is the Set of **accepting states**
-

Problem 1

Construct a DFA that recognizes the language:

$$\{w \mid w \text{ every odd position of } w \text{ is a } 1\}$$

Provide a formal definition *and* a state diagram. $\Sigma = \{0, 1\}$

Formal Definition

States

$$Q = \{q_0, q_1, q_2\}$$

- q_0 is state where next symbol is odd position
- q_1 is state where next symbol is even position
- q_2 is the sink state for all error

Language/Alphabet

$$\Sigma = \{0, 1\}$$

Transition Function δ

δ	0	1
q_0	q_2	q_1
q_1	q_0	q_0
q_2	q_2	q_2

Start State

$$q_0$$

Accepting States

$$F = \{q_0, q_1\}$$

State Diagram

provided by mermaid

- white dot is start state entry point
- highlighted state is accepted state

see figure 1

![[Screenshot 2025-02-10 at 8.30.33 PM.png|Problem 1 State Diagram]]

Problem 2

For any string $w = w_1w_2 \dots w_n$, the reverse of w , written $w^{\mathcal{R}}$, is the string w in reverse order, $w_n \dots w_2w_1$. For any language A , let $A^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in A\}$.

Prove that if A is regular, so is $A^{\mathcal{R}}$. For this, consider how, starting with any arbitrary NFA to recognize A , you'd modify the NFA to recognize $A^{\mathcal{R}}$. Once you have the idea, write a recipe that would work for any NFA.

Idea / Goal

We are looking to show that when A is regular, so is $A^{\mathcal{R}}$. Suppose A is an NFA that:

$$N = \{Q, \sum, \delta, q_0, F\}$$

Where this NFA can be represented as $L(N) = A$. We want to create an NFA that recognizes $A^{\mathcal{R}}$

Construction

We define this new NFA as M' :

$$M' = \left(Q', \sum', \delta', q'_0, F' \right)$$

It remains *relatively* unchanged compared to the original, except for a few key differences:

- we add a new **state**, q_{new} to the states set Q , where $Q' = Q \cup \{q_{new}\}$, which is essentially the new entry point for the NFA
- we add a new **accept** state, which is now $\{q_0\}$
- we reverse the transitions in the transition function:
 - if p can transition to q through a , then we reverse the transition $q \rightarrow^a p$
 - new transition function should look something like:

$$\delta'(p, (for\ a)) = \{q \in Q' \mid p\ to\ q\ through\ a\} \quad for\ all\ p$$

where “p to q through a” is representing $p \rightarrow^a q$

- epsilon transition is added for the new start state to the next state in line f :

$$q_{new} \rightarrow^{\epsilon} f$$

Formal Definition

We now define the NFA $M^{\mathcal{R}} = (Q', \Sigma', \delta', q_{new}, F')$ with the following:

States:

States Q' is Q and q_{new} which is $\rightarrow Q' = Q \cup q_{new}$

Alphabet/Finite Set of Symbols:

Alphabet stays the same as Σ so just treat it as such

Transition Function

$$\delta'(p, a) = \{q \in Q' \mid p \in \delta(q, a)\}$$

where new start state is q_0 taking path of ϵ

Start State

New start state is q_{new}

Accept States

For this theoretical NFA, the accept state is q_0

Reason This Works

This works for this example, as in the original NFA, we use a string w that is accepted if there is such path from q_0 to a state f in F that is labeled by w . When all the transitions are reversed, any of these paths now become f to q_0 labeled by the modified w , $w^{\mathcal{R}}$.

We have to add a new start state q_{new} with the transition ϵ for the states f in F because we need to allow the automaton to “start” at any of the original accept states for it to be truly reversed.

The new accepting condition comes by finding if there is a string s that $M^{\mathcal{R}}$ can take from q_{new} to q_0 that is labeled as s . Essentially, there needs to be some path in M from q_0 to some f in F that is labeled by the inverse, $s^{\mathcal{R}}$. So: $s = w^{\mathcal{R}}$ for this point w in A .

For this reason, $M^{\mathcal{R}}$ can recognize $A^{\mathcal{R}}$ exactly.

Constructing a DFA. for $A^{\mathcal{R}}$

Using this we can effectively reverse any NFA to an equivalent DFA (using our rules of subset construction, etc), to ensure that there is a DFA that can recognize $A^{\mathcal{R}}$ as required $\square$

All we need to do is take the NFA M for A , find $M^{\mathcal{R}}$ for $A^{\mathcal{R}}$ by reversing all transitions, creating new start state epsilon transition to original accept states, and making sure that the original start state is only accept state.

We can convert this to a DFA if we want to, but we know how to do this with subset construction.

Example of Reversing NFA

Original NFA M :

see figure 2

![[Screenshot 2025-02-10 at 8.30.51 PM.png|Original NFA M]]

Reversed NFA M^R with new entry point q_{new}

see figure 3

![[Screenshot 2025-02-10 at 8.31.04 PM.png|Reversed NFA M^R]]

Problem 3

Let $B_n = \{a^k \mid k \text{ is a multiple of } n\}$. Show that for each $n \geq 1$, the language B_n is regular.

Hint: Construct the base case for $n = 1$ and then construct a DFA for $n = 2$ and notice what changes. Then use what you learned to complete the proof.

Hint: If you really get stuck, and you've already produced state diagrams for the $n = 1$ and $n = 2$ cases, try producing a state diagram for the $n = 3$ case and consider what changes when going from $n = 2$ to $n = 3$.

Given/Goal

We are looking to prove that for every time $n \geq 1$, language

$$B_n = \{a^k \mid k \text{ is a multiple of } n\}$$

is a regular language.

We can do this by constructing a DFA that will recognize it. We are looking to **count** the occurrences of a in modulo n .

Cases

We have cases $1 \dots 3$ to go through, so we will construct a state diagram for each one

Base Case $n = 1$ For $n = 1$, we know that all non negative integers are multiples of 1, so:

$$B_1 = \{a^k \mid k \geq 0\} = a^*$$

This is a very simple DFA, with only a single state that is both the start and the accepting.

see figure 4

![[Screenshot 2025-02-10 at 8.32.31 PM.png|Case n=1]]

Case $n = 2$ When $n = 2$, we get:

$$B_2 = \{a^k \mid k \text{ is a multiple of } 2\}$$

This means that the language consists of all strings that have an even number of a 's.

For this we only need two states:

- q_0 : all a we have seen is even
- q_1 : all a we have seen i odd

Transitions for these two should be:

- q_0 points to q_1 when a received
- q_1 points to q_0 when a received

see figure 5

![[Screenshot 2025-02-10 at 8.32.16 PM.png|Case n=2]]

Case $n = 3$

When $n = 3$ we get:

$$B_3 = \{a^k \mid k \text{ is a multiple of } 3\}$$

Which represents when the strings received have a number of a 's that is divisible by 3

This DFA consists of 3 states:

- q_0 : when remainder is 0, multiple of 3
- q_1 : when remainder is 1 when $\div 3$
- q_2 : when remainder is 2 when $\div 3$

The transitions for these states are as follows:

- q_0 points to q_1 when a received
- q_1 points to q_2 when a received
- q_2 points to q_0 when a received

see figure 6

![[Screenshot 2025-02-10 at 8.31.54 PM.png|Case n=3]]

Expanding This out to All case $n \geq 1$

For any amount $n \geq 1$, we can construct a DFA with states:

$$q_0, q_1, \dots, q_{n-1}$$

- q_0 will always be the start and accepting state
- amount of states is dictated by number of $n - 1$

We can define a transition function for this relation with the following:

$$\delta(q_i, a) = q_{(i+1)} \mod n$$

Essentially, we can conclude that for any number n we can construct a finite automaton with a regular pattern.

Thus, B_n is a regular language for all $n \geq 1$ as required $\square$

Extra Credit Problem A

Convert the following NFA to an equivalent DFA. For *subset construction*, follow the recipe provided by Sipser Theorem 1.39 and the corresponding proof (as presented in class). Leave your construction unpruned, but be sure to indicate any unnecessary or unreachable states in the DFA thus constructed, if any.

see figure 7

![[Screenshot 2025-02-10 at 8.31.44 PM.png|Extra Credit Problem Demo]]

Listing Transitions

For state **1**:

$$\delta(1, a) = \{3\}$$

and has no b transition

For state **2**:

$$\delta(2, a) = \{1\}$$

For state **3**:

$$\delta(3, a) = \{2\}$$

$$\delta(3, b) = \{3, 2\}$$

since the b route goes to 2 and 3

Defining New DFA

1. We define the **start state** as $\{1\}$
2. Going from $\{1\}$ to:
 - $a: \delta(1, a) = \{3\}$
 - $b: \delta(1, b) = \{\emptyset\}$
3. Going from $\{3\}$ to:
 - $a: \delta(3, a) = \{2\}$
 - $b: \delta(3, b) = \{3, 2\}$
4. Going from $\{2\}$ to: (note we define this as accept state)
 - $a: \delta(2, a) = \{1\}$
 - $b: \delta(2, b) = \{\emptyset\}$
5. Going from $\{2, 3\}$ to: (also contains accept state)
 - $a: \delta(2, a) \cup \delta(3, a) = \{1, 2\}$
 - $b: \delta(2, b) \cup \delta(3, b) = \emptyset \cup \{3, 2\} \rightarrow \{3, 2\}$
6. Going from $\{1, 2\}$ to: (also contains accept state)
 - $a: \delta(1, a) \cup \delta(2, a) = \{1, 3\}$
 - $b: \delta(1, b) \cup \delta(2, b) = \emptyset \cup \emptyset \rightarrow \emptyset$
7. Going from 1, 3 to:
 - $a: \delta(1, a) \cup \delta(3, a) = \{2, 3\}$
 - $b: \delta(1, b) \cup \delta(3, b) = \emptyset \cup \{2, 3\} \rightarrow \{2, 3\}$
8. For Dead States $\emptyset$:
 - for a and $b: \delta(\emptyset, b) = \{\emptyset\}$

Constructing Table Of Transitions

DFA State	On a	On b	Accepting
$\{1\}$	$\{3\}$	$\emptyset$	No
$\{3\}$	$\{2\}$	$\{2, 3\}$	No
$\{2\}$	$\{1\}$	$\emptyset$	Yes
$\{2, 3\}$	$\{1, 2\}$	$\{2, 3\}$	Yes
$\{1, 2\}$	$\{1, 3\}$	$\emptyset$	Yes
$\{1, 3\}$	$\{2, 3\}$	$\{2, 3\}$	No

DFA State	On a	On b	Accepting
$\emptyset$	$\emptyset$	$\emptyset$	No

Note:

I could be wrong, but it seems as though all states are reachable here, so no pruning would be needed even if we wanted to.

Constructing the DFA

Defining a DFA for:

$$M = (Q, \Sigma, \delta, q_0, F)$$

- **States:** $\{1\}, \{3\}, \{2\}, \{2, 3\}, \{1, 2\}, \{1, 3\}, \emptyset$
- **Alphabet:** $\{a, b\}$
- **Start State:** 1
- **Accepting States:** $\{2\}, \{2, 3\}, \{1, 2\}$
- **Transitions:**
 - 1 goes to 3 and $\emptyset$
 - 3 goes to 2 and 2,3
 - 2 goes to 1 and $\emptyset$
 - 2,3 goes to 1,2 and 2,3
 - 1,2 goes to 1,3 and $\emptyset$
 - 1,3 goes to 2,3 and 2,3
 - $\emptyset$ goes to $\emptyset$ and $\emptyset$

Diagram

provided by mermaid

see figure 8

![[Screenshot 2025-02-10 at 8.31.26 PM.png|Resulting Constructed DFA]]